VOLUMEN 01 · MÁSTER DATA & IA

Puesta a punto

Tu Mac preparado para trabajar con datos, el repositorio del máster en GitHub y los datos de Marejada Distribución cargados en tu primera base de datos analítica.

PARA QUIÉN

Álvaro, el primer día del máster. Das por sabido lo del Volumen 1: terminal, Homebrew, Git, GitHub y VS Code.

CÓMO ESTÁ ESCRITO

Igual que el Volumen 1: cada herramienta con su porqué, cada comando desmontado, cada resultado tal y como lo verás.

INCLUYE

Cuatro tickets de bienvenida con sus soluciones, el diccionario de datos de la empresa y una chuleta de DuckDB y uv.

Preparado para Álvaro · Octubre de 2026 · Un día de trabajo

Índice

PARTE 0 · ANTES DE EMPEZAR 3

1. Cómo leer este volumen 4

2. Qué vas a instalar y por qué 6

PARTE 1 · LAS HERRAMIENTAS 7

3. Comprobar lo que ya tienes 8

4. DuckDB, tu base de datos analítica 9

5. Python con uv 13

6. VS Code para datos 15

PARTE 2 · TU PROYECTO DEL MÁSTER 16

7. El repositorio del máster 17

8. Los datos de Marejada 22

9. Cargar los datos en DuckDB 26

10. La misma base de datos desde Python 36

11. Guardar y publicar 38

12. Una vista visual (opcional) 39

TICKETS · TUS PRIMEROS ENCARGOS 40

Tickets · Volumen 01 41

Soluciones · Volumen 01 42

APÉNDICES · PARA TENER A MANO 46

A. Diccionario de datos 47

B. Chuleta 50

C. Qué viene en el Volumen 02 51

PARTE 0

Antes de empezar

Dos capítulos cortos: qué cambia respecto al Volumen 1 y qué vas a instalar hoy.

1. Cómo leer este volumen

2. Qué vas a instalar y por qué

1. Cómo leer este volumen

1.1 Lo que ya sabes

Este volumen da por sabido todo lo del Volumen 1: moverte por la terminal, Homebrew, Git y GitHub, y VS Code. Si en algún momento un comando de esos te suena raro, vuelve a su capítulo allí. Aquí no se repite.

Las marcas visuales son las mismas. Lo que tú escribes en la terminal, en recuadro claro:

TÚ ESCRIBES

```
cd ~/master-data-ia
```

Lo que responde el ordenador, en recuadro oscuro, y los errores con la marca roja:

RESULTADO

```
/Users/alvaro/master-data-ia
```

1.2 Las marcas nuevas

A partir de ahora vas a escribir en tres sitios distintos, y cada uno tiene su recuadro. El primero ya lo conoces: la terminal. El segundo es **SQL dentro de DuckDB**, con su etiqueta morada:

SQL · EN DUCKDB

```
SELECT 'Hola, Marejada' AS saludo;
```

RESULTADO

```
┌─────────────────┐
│     saludo      │
│    varchar      │
├─────────────────┤
│ Hola, Marejada  │
└─────────────────┘
```

Fíjate en que debajo aparece el resultado. En los recuadros de SQL de estos volúmenes, el resultado no está escrito a mano: es la salida real de ejecutar esa consulta sobre los datos que vas a cargar hoy. Si a ti te sale distinto, algo has escrito diferente, y merece la pena averiguar qué.

El tercer sitio es **Python en un notebook**, con su etiqueta verde:

PYTHON · EN TU NOTEBOOK

```
print("Hola desde Python")
```

Y cuando haya que crear un fichero con un contenido concreto, aparece así, con el nombre del fichero en la etiqueta:

FICHERO · sql/ejemplo.sql

```
-- Esto es el contenido de un fichero
SELECT 42;
```

Por último, los **tickets**: los encargos que te llegan de la gente de la empresa. Los verás al final de cada volumen. Y después de cada solución, la **revisión de Elena**, tu responsable, con lo que te diría al revisarla.

1.3 Lo que tendrás al acabar el día

AL TERMINAR	PARA QUÉ TE SIRVE
DuckDB instalado y entendido	Es donde vas a practicar todo el SQL del máster.
Python 3.12 con uv y los paquetes de datos	Lo usarás desde el Volumen 05 y ya hoy para comprobar que todo encaja.
Un repositorio `master-data-ia` en GitHub	Tu portfolio. Cada ticket que resuelvas acabará ahí.
Los datos de Marejada en `datos/`	Los mismos durante todo el máster.
Una base de datos `db/marejada.duckdb` con los datos cargados	El punto de partida del Volumen 02.
Un script que reconstruye la base de datos desde cero	Para que todo sea repetible, que es la primera regla del trabajo con datos.

2. Qué vas a instalar y por qué

HERRAMIENTA	QUÉ ES, EN UNA FRASE	¿YA LA TIENES?
Homebrew	El instalador de programas por terminal.	Sí, del Volumen 1
Git y gh	Historial de versiones y conexión con GitHub.	Sí, del Volumen 1
VS Code	El editor. Hoy le añadimos extensiones para datos.	Sí, del Volumen 1
DuckDB	Una base de datos analítica que vive en un fichero de tu Mac.	No: capítulo 4
uv	El programa que instala Python y gestiona los paquetes de cada proyecto.	No: capítulo 5
Python 3.12	El lenguaje de los datos. Lo instala uv.	No: capítulo 7
Paquetes de datos	duckdb, pandas, jupyter, openpyxl y matplotlib.	No: capítulo 7

Lo que **no** vas a instalar hoy, aunque lo necesitarás: Windows y Power BI Desktop (Volumen 07), las cuentas de Fabric, Databricks y AWS (al empezar cada bloque, como explica la guía). Instalar las cosas cuando se necesitan evita tener el Mac lleno de programas a medio configurar.

> **IDEA CLAVE**
>
> Hoy no se aprende SQL ni Python: se monta el taller. Pero se monta entendiendo cada pieza, porque este taller es el mismo que vas a usar cinco meses.

PARTE 1

Las herramientas

Comprobar lo que tienes, instalar DuckDB y uv, y preparar VS Code para trabajar con datos.

3. Comprobar lo que ya tienes

4. DuckDB, tu base de datos analítica

5. Python con uv

6. VS Code para datos

3. Comprobar lo que ya tienes

3.1 Que todo responde

Abre la Terminal y comprueba las cuatro herramientas del Volumen 1:

TÚ ESCRIBES

```
brew --version
git --version
gh --version
code --version
```

Las cuatro tienen que contestar con un número de versión. Si alguna dice `command not found`, vuelve al capítulo 3 del Volumen 1 antes de seguir.

3.2 Poner Homebrew al día

Homebrew no se actualiza solo. Antes de instalar nada nuevo, conviene ponerlo al día:

TÚ ESCRIBES

```
brew update
brew upgrade
```

DESMONTANDO EL COMANDO

`brew update && brew upgrade`

`brew update`	Descarga la lista actualizada de programas y versiones disponibles. No instala nada: solo se entera de qué hay nuevo.
`brew upgrade`	Actualiza todo lo que tengas instalado a su última versión.

La diferencia importa: `update` es leer el catálogo nuevo de la tienda; `upgrade` es cambiar lo que tienes en casa por lo del catálogo nuevo. Puede tardar unos minutos.

4. DuckDB, tu base de datos analítica

4.1 Dos tipos de bases de datos

En ASIR viste bases de datos como MySQL o PostgreSQL: un servidor que siempre está en marcha y al que se conectan las aplicaciones para guardar y leer datos. Esas bases de datos están pensadas para un trabajo concreto: **muchas operaciones pequeñas a la vez**. Un cliente hace un pedido: se inserta una fila. Otro cambia su dirección: se modifica una fila. Miles de veces por minuto, cada una rapidísima.

El trabajo de un analista de datos es el contrario: **pocas consultas, pero enormes**. "¿Cuánto hemos vendido por provincia y mes en los últimos tres años?" obliga a leer cientos de miles de filas para devolver unas pocas.

Son dos oficios tan distintos que tienen nombre propio:

	OLTP (TRANSACCIONAL)	OLAP (ANALÍTICA)
Viene de	Online Transaction Processing	Online Analytical Processing
Pregunta típica	"Guarda este pedido"	"¿Cuánto vendimos por provincia y mes?"
Filas que toca cada operación	Una o pocas	Cientos de miles o millones
Quién la usa	Las aplicaciones: el ERP, la tienda web	Los analistas, los informes, los modelos
Comparación	La caja registradora de una tienda	El libro de cuentas de fin de año
Ejemplos	MySQL, PostgreSQL, SQL Server	DuckDB, Fabric Warehouse, Databricks SQL, Redshift

4.2 Filas o columnas: el truco de las bases analíticas

La diferencia de fondo está en cómo guardan los datos en el disco. Imagina la tabla de líneas de pedido con tres columnas. Una base de datos transaccional la guarda **por filas**: todos los datos de una línea, juntos.

```
Por filas (OLTP):      [1, 1192, 41.65] [2, 1073, 95.85] [3, 1020, 95.49] ...

Por columnas (OLAP):   [1, 2, 3, ...]
                       [1192, 1073, 1020, ...]
                       [41.65, 95.85, 95.49, ...]
```

Si quieres sumar los precios de 381.000 líneas, la versión por filas obliga a leerlo todo, incluidas las columnas que no te interesan. La versión **por columnas** lee solo la columna del precio. Y como los valores de una misma columna se parecen mucho entre sí, se comprimen muy bien, así que además ocupa menos y se lee más rápido.

Ese es el principio de todas las herramientas analíticas que vas a ver en el máster: DuckDB, los ficheros Parquet, Delta Lake en Databricks, el Warehouse de Fabric o Redshift en AWS. Todas guardan por columnas.

4.3 Por qué DuckDB

CARACTERÍSTICA	QUÉ SIGNIFICA PARA TI
Gratuito y de código abierto	Sin licencias ni pruebas que caducan.
Sin servidor	No hay nada que arrancar ni administrar. La base de datos es un fichero, como un documento.
Lee ficheros directamente	Puedes consultar un CSV, un Excel o un JSON sin cargarlo antes.
Muy rápido en un portátil	381.000 líneas son poca cosa para él.
SQL muy parecido al estándar	Lo que aprendas te sirve casi igual en Fabric y Databricks. Las diferencias las veremos en el Volumen 02.
Se usa desde la terminal y desde Python	El mismo fichero de base de datos, desde los dos sitios.

Se usa de verdad en empresas, sobre todo para análisis en local, para procesar ficheros y como motor dentro de otras herramientas. Pero aquí lo importante es otra cosa: es el mejor sitio para aprender SQL analítico sin depender de ninguna cuenta en la nube.

4.4 Instalarlo

TÚ ESCRIBES

```
brew install duckdb
duckdb --version
```

RESULTADO

```
v1.5.x (...)
```

El número exacto da igual mientras empiece por 1.5 o más. Estos volúmenes están probados con la versión 1.5.

4.5 Tu primera sesión

Escribe solo el nombre del programa:

TÚ ESCRIBES

```
duckdb
```

RESULTADO

```
DuckDB v1.5.x (...)
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
D
```

Esa `D` del final es el **prompt de DuckDB**: igual que el `%` de la terminal, significa "estoy esperando que escribas". Pero ojo, porque ahora ya no estás hablando con la terminal: estás hablando con DuckDB, y DuckDB entiende SQL, no comandos de terminal. Si escribes `ls`, no sabrá qué es.

Prueba tu primera consulta:

SQL · EN DUCKDB

```
SELECT 6 * 7 AS respuesta;
```

RESULTADO

```
┌──────────┐
│ respuesta │
│   int32   │
├──────────┤
│        42 │
└──────────┘
```

Ya conoces `SELECT` de ASIR. Lo que te conviene fijarte es en el formato de la respuesta, porque lo vas a ver miles de veces:

DESMONTANDO LA CONSULTA

`SELECT 6 * 7 AS respuesta;`

`respuesta`	La cabecera: el nombre de la columna. Lo has elegido tú con `AS`.
`int32`	El tipo de dato de esa columna: un número entero. DuckDB siempre te dice el tipo de cada columna, y te va a salvar de muchos errores.
`42`	El valor.
`;`	El punto y coma marca el final de la consulta. Sin él, DuckDB no la ejecuta (ver 4.7).

4.6 Los comandos con punto

Además de SQL, DuckDB entiende unos comandos propios que empiezan por un punto. No son SQL: son órdenes para el programa en sí. Los que vas a usar:

COMANDO	QUÉ HACE
`.help`	Lista todos los comandos con punto.
`.tables`	Lista las tablas de la base de datos.
`.timer on`	A partir de ahora, te dice cuánto ha tardado cada consulta.
`.read fichero.sql`	Ejecuta todas las consultas que haya en un fichero.
`.mode`	Cambia el formato de los resultados (por defecto, el de las cajas que ves).
`.quit`	Salir. También vale `Ctrl` + `D`.

Sal de DuckDB con `.quit`. Vuelves a la terminal: el prompt vuelve a ser `%`.

4.7 En memoria o en un fichero

Fíjate en la frase del arranque: *Connected to a transient in-memory database*. "Transitoria y en memoria": todo lo que crees en esa sesión desaparece al salir. Es útil para pruebas rápidas, pero no para trabajar.

Para trabajar de verdad, se arranca DuckDB diciéndole qué fichero usar:

TÚ ESCRIBES

```
duckdb prueba.duckdb
```

Si el fichero no existe, lo crea. Si existe, lo abre. Todo lo que crees dentro se guarda en ese fichero y seguirá ahí la próxima vez. En el capítulo 9 crearás así la base de datos de Marejada. De momento, sal con `.quit` y borra el fichero de prueba:

TÚ ESCRIBES

```
rm prueba.duckdb
```

4.8 Errores típicos

Pulsar Intro y que no pase nada. Si escribes una consulta sin punto y coma y pulsas Intro, DuckDB no la ejecuta: entiende que la consulta continúa en la línea siguiente y cambia el prompt para indicártelo. Escribe `;` y pulsa Intro otra vez. Esto es una ventaja, no un fallo: te permite escribir consultas largas en varias líneas.

Escribir comandos de terminal dentro de DuckDB. Si ves un error de sintaxis al escribir `cd` o `ls`, es que estás dentro de DuckDB (prompt `D`), no en la terminal (prompt `%`). Sal con `.quit`.

Equivocarse de comillas. En SQL los textos van entre comillas **simples**: `'Santander'`. Las comillas dobles son para nombres de columnas o tablas: `"Base imponible (€)"`. Si pones un texto entre comillas dobles, DuckDB buscará una columna con ese nombre:

SQL · EN DUCKDB

```
SELECT "Santander";
```

ERROR

```
Binder Error: Referenced column "Santander" was not found because the FROM clause is missing
```

Léelo con calma, porque es muy preciso: "no he encontrado la columna `Santander`, porque ni siquiera hay un `FROM`". DuckDB ha entendido que `"Santander"` era el nombre de una columna, y una columna solo puede venir de una tabla. Con comillas simples, `SELECT 'Santander';`, sería un texto y funcionaría. Y la flecha `^` de la última línea señala el punto exacto donde se ha perdido, como en los errores de JavaScript del Volumen 1.

> **IDEA CLAVE**
>
> DuckDB es una base de datos analítica: guarda por columnas, vive en un fichero y lee CSV, Excel y JSON directamente. El prompt `D` significa que hablas SQL, no terminal. Cada consulta termina en `;`, los textos van entre comillas simples y `.quit` te saca.

5. Python con uv

5.1 Por qué Python en un máster de datos

SQL es el idioma para preguntar a los datos que ya están en una base de datos. Python es el idioma para todo lo demás: leer datos de una API, limpiar ficheros que SQL no entiende bien, automatizar tareas, y sobre todo, todo el machine learning y la IA del máster. Desde el Volumen 05 lo usarás a diario; hoy solo lo instalamos y comprobamos que se entiende con DuckDB.

5.2 El problema que resuelve un entorno virtual

Python funciona con **paquetes**: código escrito por otras personas que instalas para no reinventarlo. Pandas, por ejemplo, es el paquete para manejar tablas. El problema aparece cuando tienes varios proyectos:

- El proyecto A necesita pandas 2.2.
- El proyecto B, que empezaste hace un año, necesita pandas 1.5, porque con la 2.2 algo deja de funcionar.

Si los paquetes se instalan para todo el ordenador, solo puede haber una versión, y uno de los dos proyectos se rompe. La solución es que **cada proyecto tenga su propia caja de herramientas**, separada de las demás. Esa caja se llama **entorno virtual**: una carpeta dentro del proyecto (casi siempre llamada `.venv`) con su propio Python y sus propios paquetes.

Es lo mismo que el `node_modules` que viste con Node en el Volumen 1: cada proyecto lleva sus propias dependencias.

> **NO USES EL PYTHON DEL SISTEMA**
>
> Tu Mac trae un `python3` de serie. No lo uses para tus proyectos: es una versión que macOS necesita para sí mismo, a menudo antigua, y si instalas paquetes ahí puedes acabar con conflictos difíciles de arreglar. Todo el Python del máster irá dentro de entornos virtuales gestionados por uv.

5.3 Qué es uv

Durante años, preparar Python requería tres o cuatro herramientas distintas: una para instalar versiones de Python, otra para crear entornos virtuales, otra para instalar paquetes y otra para recordar qué versiones exactas usaba cada proyecto. **uv** las sustituye todas con un único programa, y además es muchísimo más rápido. Es gratuito y es hoy una de las formas más usadas de gestionar proyectos de Python.

LO QUE NECESITAS	CON UV
Instalar una versión concreta de Python	`uv` la descarga sola cuando hace falta
Crear el entorno virtual de un proyecto	`uv` lo crea solo al añadir el primer paquete
Instalar un paquete en el proyecto	`uv add pandas`
Recordar las versiones exactas	`uv` las apunta en `uv.lock`
Ejecutar algo con el Python del proyecto	`uv run ...`

5.4 Instalarlo

TÚ ESCRIBES

```
brew install uv
uv --version
```

RESULTADO

```
uv 0.x.y (...)
```

Ya está. Python en sí lo instalará uv en el capítulo 7, cuando creemos el proyecto.

6. VS Code para datos

VS Code no sabe trabajar con notebooks de Python de serie: se le añaden **extensiones**, que son complementos que amplían lo que sabe hacer. Se pueden instalar desde la propia aplicación, pero por terminal es más rápido y queda escrito qué has instalado:

TÚ ESCRIBES

```
code --install-extension ms-python.python
code --install-extension ms-toolsai.jupyter
code --install-extension mechatroner.rainbow-csv
```

DESMONTANDO EL COMANDO

`code --install-extension ms-python.python`

`code`	El comando de VS Code que activaste en el Volumen 1.
`--install-extension`	La opción: instalar una extensión.
`ms-python.python`	El identificador de la extensión: `editor.nombre` . `ms` es Microsoft.

EXTENSIÓN	PARA QUÉ
Python (Microsoft)	Que VS Code entienda Python: colores, sugerencias, encontrar tu entorno virtual.
Jupyter (Microsoft)	Abrir y ejecutar notebooks dentro de VS Code.
Rainbow CSV	Colorea cada columna de un CSV de un color. Con ficheros separados por punto y coma, es la diferencia entre entender algo y no entender nada.

DÓNDE VER TUS EXTENSIONES

En VS Code, el icono de los cuatro cuadraditos de la barra izquierda (o `Cmd` + `Shift` + `X`) lista las extensiones instaladas.

PARTE 2

Tu proyecto del máster

El repositorio, los datos de Marejada y su carga en DuckDB, comprobada desde la terminal y desde Python.

7. El repositorio del máster

8. Los datos de Marejada

9. Cargar los datos en DuckDB

10. La misma base de datos desde Python

11. Guardar y publicar

12. Una vista visual (opcional)

7. El repositorio del máster

7.1 La estructura

Todo el máster va a vivir en una sola carpeta, que será también un repositorio de Git publicado en GitHub. Esta es la estructura:

```
master-data-ia/
├── README.md          ← la portada del repositorio en GitHub
├── .gitignore         ← lo que nunca se sube
├── pyproject.toml     ← el proyecto de Python (lo crea uv)
├── uv.lock            ← las versiones exactas de los paquetes (lo crea uv)
├── .venv/             ← el entorno virtual (lo crea uv; no se sube)
├── datos/             ← los ficheros de Marejada (no se suben)
├── db/                ← la base de datos DuckDB (no se sube)
├── sql/               ← tus consultas, una carpeta por volumen
├── notebooks/         ← los notebooks de Python
└── entregables/       ← lo que "entregas" en cada ticket
```

La idea que hay detrás: separar **lo que tú escribes** (consultas, notebooks, documentos) de **lo que se puede regenerar** (la base de datos, el entorno virtual) y de **lo que viene de fuera** (los datos de la empresa). Solo lo primero va a GitHub.

7.2 Crear la carpeta y el repositorio

TÚ ESCRIBES

```
cd ~
mkdir -p master-data-ia/{datos,db,sql,notebooks,entregables}
cd master-data-ia
git init -b main
```

Es la expansión de llaves y el `-p` del capítulo 6 del Volumen 1: una sola línea crea la carpeta principal y las cinco de dentro.

7.3 Git no guarda carpetas vacías

Aquí hay algo que sorprende la primera vez. Git no guarda carpetas: guarda ficheros, y las carpetas solo existen para él como parte de la ruta de un fichero. Una carpeta vacía, para Git, no existe. Si subieras ahora el repositorio, en GitHub no aparecerían `sql`, `notebooks` ni `entregables`.

La costumbre universal para conservarlas es meter dentro un fichero vacío llamado `.gitkeep`. El nombre no tiene nada de especial para Git; es solo una convención que todo el mundo entiende:

TÚ ESCRIBES

```
touch sql/.gitkeep notebooks/.gitkeep entregables/.gitkeep
```

7.4 Lo que nunca se sube

Crea el `.gitignore` antes de nada, como aprendiste en el Volumen 1:

TÚ ESCRIBES

```
code .gitignore
```

FICHERO · .gitignore

```
# macOS
.DS_Store

# Python
.venv/
__pycache__/
.ipynb_checkpoints/

# Datos y bases de datos: se regeneran o vienen de fuera
datos/
db/
*.duckdb
*.duckdb.wal
```

Por qué cada bloque:

LÍNEA	POR QUÉ NO SE SUBE
`.venv/`	Pesa cientos de megas y se regenera en segundos con `uv sync` a partir de `uv.lock`.
`__pycache__/`, `.ipynb_checkpoints/`	Ficheros temporales que crean Python y Jupyter.
`datos/`	Son datos de la empresa. En una empresa real, subir datos de clientes a GitHub es una fuga de datos personales. Aquí son inventados, pero la costumbre se coge desde el primer día.
`db/`, `*.duckdb`, `*.duckdb.wal`	La base de datos se reconstruye con tu script (capítulo 9). El `.wal` es un fichero temporal que DuckDB usa mientras escribe.

IDEA CLAVE

En un repositorio de datos se sube el código que produce los resultados, nunca los datos. Si alguien clona tu repositorio, con los datos originales y tus scripts debe poder reconstruirlo todo.

7.5 El proyecto de Python

Ahora le dices a uv que esta carpeta es un proyecto de Python, con Python 3.12:

TÚ ESCRIBES

```
uv init --bare --python 3.12
```

RESULTADO

```
Initialized project `master-data-ia`
```

DESMONTANDO EL COMANDO

`uv init --bare --python 3.12`

`uv init`	Convierte la carpeta en un proyecto de Python.
`--bare`	"Mínimo": crea solo el fichero `pyproject.toml`, sin ficheros de ejemplo que no vas a usar.
`--python 3.12`	La versión de Python del proyecto. La 3.12 es estable y compatible con todo lo que usaremos.

Y añade los paquetes que vas a necesitar en el Bloque 0:

TÚ ESCRIBES

```
uv add duckdb pandas jupyter ipykernel openpyxl matplotlib
```

RESULTADO

```
Using CPython 3.12.x
Creating virtual environment at: .venv
Resolved 100 packages in 1.2s
Prepared 98 packages in 6.3s
Installed 98 packages in 410ms
 + duckdb==1.5.x
 + pandas==...
 ...
```

Las cifras exactas te saldrán distintas; lo que importa es que termine sin errores. Esto ha hecho cuatro cosas de golpe: descargar Python 3.12 (si no lo tenías), crear el entorno virtual `.venv`, instalar los seis paquetes y todos los paquetes de los que ellos dependen (de ahí que sean casi cien), y apuntarlo todo en dos ficheros:

FICHERO	QUÉ CONTIENE	¿SE SUBE A GITHUB?
`pyproject.toml`	Lo que tú has pedido: "este proyecto usa Python 3.12, duckdb, pandas..."	Sí
`uv.lock`	Las versiones exactas de los cien paquetes que se han instalado	Sí
`.venv/`	Los paquetes en sí	No

Con `pyproject.toml` y `uv.lock`, cualquiera (o tú en otro ordenador) recrea exactamente el mismo entorno con un solo comando: `uv sync`.

PAQUETE	PARA QUÉ
duckdb	Usar DuckDB desde Python.
pandas	Trabajar con tablas en Python. Lo verás a fondo en el Volumen 06.
jupyter, ipykernel	Los notebooks, y que VS Code pueda usar el Python de este proyecto en ellos.
openpyxl	Leer y escribir ficheros Excel.
matplotlib	Gráficos.

Comprueba que todo responde:

TÚ ESCRIBES

```
uv run python -c "import duckdb, pandas; print(duckdb.__version__, pandas.__version__)"
```

RESULTADO

```
1.5.x 3.0.x
```

DESMONTANDO EL COMANDO

`uv run python -c "..."`

`uv run`	"Ejecuta lo que sigue usando el entorno de este proyecto". Así no tienes que activar nada a mano.
`python`	El Python del proyecto (el 3.12 del `.venv`), no el del sistema.
`-c "..."`	Ejecuta el código que va entre comillas, sin necesidad de un fichero.

7.6 El README

TÚ ESCRIBES

```
code README.md
```

FICHERO · README.md

```
# Máster Data & IA

Mi formación en ingeniería de datos e IA: SQL, Python, Power BI,
Microsoft Fabric, Databricks, AWS, machine learning, deep learning
e IA generativa, con cuatro certificaciones oficiales como objetivo.

Todo el trabajo se hace sobre los datos de una empresa ficticia,
Marejada Distribución, S.L., una distribuidora de bebidas y
alimentación para hostelería en el norte de España.

## Estructura

- `sql/` — consultas, una carpeta por volumen
- `notebooks/` — análisis en Python
- `entregables/` — resultados entregados en cada ticket

## Cómo reproducirlo

1. `uv sync` para crear el entorno de Python.
2. Copiar los datos de la empresa en `datos/`.
3. `duckdb db/marejada.duckdb < sql/00_cargar_bronze.sql`
```

El punto 3 todavía no funciona: el script lo escribes en el capítulo 9. Un README dice cómo se usa el proyecto, y tiene sentido escribirlo pensando en cómo va a quedar.

7.7 Primer commit y publicación

TÚ ESCRIBES

```
git status
```

RESULTADO

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        README.md
        entregables/
        notebooks/
        pyproject.toml
        sql/
        uv.lock

nothing added to commit but untracked files present (use "git add" to track)
```

Lee la lista con atención, porque es una comprobación: **no aparecen** `.venv/`, `datos/` ni `db/`. El `.gitignore` funciona. Ahora sí:

TÚ ESCRIBES

```
git add .
git commit -m "chore: estructura inicial del proyecto del máster"
gh repo create master-data-ia --public --source=. --push
```

Es exactamente lo del capítulo 17 del Volumen 1. Abre el repositorio en el navegador con `gh repo view --web` y comprueba que el README aparece como portada.

8. Los datos de Marejada

8.1 Descomprimir

Junto con este volumen tienes el fichero `datos_marejada_v1.zip`. Descárgalo; irá a tu carpeta de Descargas. Desde la carpeta del proyecto:

TÚ ESCRIBES

```
cd ~/master-data-ia
unzip ~/Downloads/datos_marejada_v1.zip -d .
```

DESMONTANDO EL COMANDO

`unzip ~/Downloads/datos_marejada_v1.zip -d .`

`unzip`	Descomprime un fichero .zip.
`~/Downloads/datos_marejada_v1.zip`	El fichero que descomprimes.
`-d .`	Dónde dejar el contenido. El punto, como siempre, es "aquí". El zip ya trae dentro la carpeta `datos/`, así que todo cae en su sitio.

SI DICE QUE YA EXISTE

Como `datos/` ya existía (vacía), `unzip` podría preguntarte si quieres sobrescribir algún fichero con algo como `replace datos/LEEME.md? [y]es, [n]o, [A]ll...`. Responde `A` (todo).

8.2 Ver qué ha llegado

TÚ ESCRIBES

```
find datos -type f | sort
```

RESULTADO

```
datos/LEEME.md
datos/almacen/inventario_2026-09-30.csv
datos/contabilidad/facturacion_mensual_2024-2026.xlsx
datos/erp/clientes.csv
datos/erp/lineas_pedido.csv
datos/erp/pedidos.csv
datos/erp/productos.csv
datos/rrhh/comerciales.csv
datos/web/pedidos_web.json
```

`find` ya lo conoces; `-type f` hace que liste solo ficheros (*files*), no carpetas. Y cuánto ocupa cada carpeta:

TÚ ESCRIBES

```
du -sh datos/*
```

RESULTADO

```
 36K    datos/almacen
 16K    datos/contabilidad
 20M    datos/erp
4.0K    datos/LEEME.md
8.0K    datos/rrhh
5.7M    datos/web
```

DESMONTANDO EL COMANDO

`du -sh datos/*`

`du`	*Disk usage*: cuánto ocupa algo en el disco.
`-s`	*Summarize*: un total por cada elemento, sin desglosar todo lo de dentro.
`-h`	*Human-readable*: en K, M o G, en lugar de en bloques.
`datos/*`	Cada cosa que hay dentro de `datos`.

Unos 25 MB en total. Para un analista es un volumen pequeño; para Excel ya sería incómodo. El ERP es casi todo.

8.3 Mirar dentro sin abrir nada

Antes de cargar un fichero de datos en ningún sitio, se mira. La forma más rápida es `head`, del Volumen 1:

TÚ ESCRIBES

```
head -3 datos/erp/lineas_pedido.csv
```

RESULTADO

```
linea_id;pedido_id;num_linea;producto_id;cantidad;precio_unitario;descuento_pct
1;2024000001;1;1192;6;41,65;0
2;2024000001;2;1073;2;95,85;0
```

Tres líneas bastan para descubrir tres cosas que vas a necesitar:

1. **La primera línea es la cabecera**: los nombres de las columnas.
2. **El separador es el punto y coma**, no la coma. Es lo habitual en programas configurados en español, precisamente por lo siguiente.
3. **Los decimales llevan coma**: `41,65`. Por eso el separador no puede ser la coma: no se sabría si `41,65` son dos columnas o un número.

Mira también los pedidos:

TÚ ESCRIBES

```
head -3 datos/erp/pedidos.csv
```

RESULTADO

```
pedido_id;fecha_pedido;cliente_id;comercial_id;canal;estado;fecha_entrega;almacen;gastos_envio;observaci
ones
2024000001;01/01/2024 08:03;10562;4;Comercial;Entregado;04/01/2024;Guarnizo;0,00;Factura a nombre de la
S.L.
2024000002;01/01/2024 08:21;11114;2;Comercial;Entregado;02/01/2024;Guarnizo;0,00;
```

Una cuarta cosa: **las fechas están en formato español**, día/mes/año. `04/01/2024` es el 4 de enero, no el 1 de abril. Un programa configurado en inglés lo leería al revés, y sin dar ningún error.

> **IDEA CLAVE**
>
> Los datos que llegan de un ERP español casi siempre traen punto y coma como separador, coma decimal y fechas día/mes/año. Mirar las tres primeras líneas con `head` antes de cargar nada te ahorra horas.

8.4 Contar filas

TÚ ESCRIBES

```
wc -l datos/erp/*.csv
```

RESULTADO

```
   1213 datos/erp/clientes.csv
 381159 datos/erp/lineas_pedido.csv
  77449 datos/erp/pedidos.csv
    236 datos/erp/productos.csv
 460057 total
```

Apúntalo, porque te servirá para comprobar la carga. Y recuerda que la primera línea de cada fichero es la cabecera: los clientes son 1.212, no 1.213.

8.5 La codificación de caracteres

Ahora mira el fichero de Recursos Humanos:

TÚ ESCRIBES

```
head -3 datos/rrhh/comerciales.csv
```

RESULTADO

```
comercial_id,nombre,apellidos,email,zona,puesto,jefe_id,fecha_alta,fecha_baja
1,��igo,Sainz Ruiz,inigo.sainz@marejada-distribucion.es,,Director comercial,,2015-03-02,
2,Raquel,Fern�ndez Cobo,raquel.fernandez@marejada-distribucion.es,Cantabria Centro,Jefa de ventas
Cantabria,1,2016-09-12,
```

`Íñigo` se ve como `��igo` y `Fernández` como `Fern�ndez`. El fichero no está roto: está escrito en otra **codificación**.

Un ordenador no guarda letras, guarda números. Una codificación es la tabla que dice qué número corresponde a cada letra. Para las letras sin acentos, casi todas las tablas coinciden; para la ñ, las tildes o el €, no:

CODIFICACIÓN	CÓMO GUARDA LA "Ñ"	DÓNDE LA VERÁS
UTF-8	Dos bytes: `C3 B1`	El estándar actual. Tu Mac, la web, Python, DuckDB.
Latin-1 (ISO-8859-1)	Un byte: `F1`	Programas antiguos, sobre todo de Windows: nóminas, ERPs, exportaciones de Excel.

Tu terminal espera UTF-8. Cuando encuentra el byte `F1` suelto, que en UTF-8 no significa nada, pinta el símbolo `�`, que quiere decir literalmente "aquí hay algo que no sé leer". Para preguntarle al Mac qué codificación cree que tiene cada fichero:

TÚ ESCRIBES

```
file -I datos/rrhh/comerciales.csv datos/erp/clientes.csv
```

RESULTADO

```
datos/rrhh/comerciales.csv: text/csv; charset=iso-8859-1
datos/erp/clientes.csv:     text/plain; charset=utf-8
```

`iso-8859-1` es Latin-1. Es un caso clásico: el programa de nóminas de Marejada es antiguo. Cuando carguemos este fichero habrá que decirle a DuckDB en qué codificación está.

9. Cargar los datos en DuckDB

9.1 La idea de las capas

Antes de cargar nada, una decisión de diseño que vas a ver en todo el máster. Los datos no se cargan y se limpian de golpe: se organizan en **capas**, y cada capa tiene una función.

CAPA	QUÉ CONTIENE	REGLA
Bronze (bronce)	Los datos tal cual llegan de cada sistema, solo con los tipos bien leídos.	No se corrige nada. Es la copia fiel del origen.
Silver (plata)	Los datos limpios: duplicados quitados, nombres normalizados, errores resueltos.	Aquí van las reglas de negocio.
Gold (oro)	Los datos preparados para un uso concreto: el informe comercial, el modelo de predicción.	Hechos a la medida de quien los usa.

Se llama **arquitectura medallón** y es la forma estándar de organizar los datos en Databricks y en Fabric. Lo verás a fondo en los bloques 2 y 3. ¿Por qué no limpiar directamente al cargar? Porque si un día descubres que una limpieza estaba mal, necesitas volver al dato original. Si lo corregiste al cargarlo, lo has perdido.

Hoy solo vas a construir la capa bronze. La silver la construyes en el Volumen 02.

9.2 Crear la base de datos

Desde la carpeta del proyecto, arranca DuckDB con el fichero de la base de datos:

TÚ ESCRIBES

```
cd ~/master-data-ia
duckdb db/marejada.duckdb
```

`db/marejada.duckdb` no existe todavía, así que DuckDB lo crea. A partir de ahora, siempre que trabajes con los datos de Marejada abrirás DuckDB así, **desde la carpeta del proyecto**. Importa por lo siguiente: dentro de DuckDB, las rutas a los ficheros son relativas a la carpeta desde la que lo arrancaste, igual que en la terminal.

9.3 Leer un CSV sin cargarlo

DuckDB puede consultar un fichero directamente, poniendo su ruta donde iría el nombre de una tabla:

SQL · EN DUCKDB

```
SELECT * FROM 'datos/erp/clientes.csv' LIMIT 3;
```

RESULTADO

```
┌────────────┬─────────────┬──────────────────┬───┬─────────┬──────────────────┬──────────────────┐
│ cliente_id │     nif     │ nombre_comercial │ … │ activo  │ limite_credito   │ descuento_habitual │
│   int64    │   varchar   │     varchar      │ … │ varchar │      int64       │       int64        │
├────────────┼─────────────┼──────────────────┼───┼─────────┼──────────────────┼──────────────────┤
│      10001 │ B39007919   │ Bar La Ola       │ … │ S       │             1500 │                5 │
│      10002 │ B-39015838  │ Bar La Ría       │ … │ S       │             5000 │               10 │
│      10003 │ A39023757   │ Bar Bitácora     │ … │ S       │             5000 │               10 │
└────────────┴─────────────┴──────────────────┴───┴─────────┴──────────────────┴──────────────────┘
  3 rows                                                    15 columns (6 shown)
```

Funciona sin que le hayas dicho nada del punto y coma. DuckDB lleva un **detector de formato** que lee el principio del fichero y adivina el separador, si hay cabecera y el tipo de cada columna. Fíjate en el pie: *15 columns (7 shown)*. La tabla no cabe en el ancho, así que DuckDB oculta las del centro (las de la columna `…`). No es un error.

9.4 DESCRIBE: lo que DuckDB ha entendido

Que algo se lea sin errores no significa que se haya leído bien. La pregunta importante es **de qué tipo** ha decidido DuckDB que es cada columna. Para eso está `DESCRIBE`:

SQL · EN DUCKDB

```
DESCRIBE SELECT * FROM 'datos/erp/productos.csv';
```

RESULTADO

```
┌─────────────────┬─────────────┬─────────┬─────────┬─────────┬─────────┐
│   column_name   │ column_type │  null   │   key   │ default │  extra  │
│     varchar     │   varchar   │ varchar │ varchar │ varchar │ varchar │
├─────────────────┼─────────────┼─────────┼─────────┼─────────┼─────────┤
│ producto_id     │ BIGINT      │ YES     │ NULL    │ NULL    │ NULL    │
│ sku             │ VARCHAR     │ YES     │ NULL    │ NULL    │ NULL    │
│ nombre          │ VARCHAR     │ YES     │ NULL    │ NULL    │ NULL    │
│ categoria       │ VARCHAR     │ YES     │ NULL    │ NULL    │ NULL    │
│ subcategoria    │ VARCHAR     │ YES     │ NULL    │ NULL    │ NULL    │
│ marca           │ VARCHAR     │ YES     │ NULL    │ NULL    │ NULL    │
│ formato         │ VARCHAR     │ YES     │ NULL    │ NULL    │ NULL    │
│ precio_lista    │ VARCHAR     │ YES     │ NULL    │ NULL    │ NULL    │
│ coste_unitario  │ VARCHAR     │ YES     │ NULL    │ NULL    │ NULL    │
│ iva_pct         │ BIGINT      │ YES     │ NULL    │ NULL    │ NULL    │
│ activo          │ VARCHAR     │ YES     │ NULL    │ NULL    │ NULL    │
│ fecha_alta      │ DATE        │ YES     │ NULL    │ NULL    │ NULL    │
│ fecha_baja      │ DATE        │ YES     │ NULL    │ NULL    │ NULL    │
└─────────────────┴─────────────┴─────────┴─────────┴─────────┴─────────┘
  13 rows                                                  6 columns
```

Repasa la columna `column_type`. La mayoría están bien: `producto_id` es `BIGINT` (un entero grande), las fechas son `DATE`. Pero `precio_lista` y `coste_unitario` son `VARCHAR`: **texto**. DuckDB ha visto `102,00` y, como en su idioma por defecto el decimal es un punto, ha concluido que eso no es un número.

¿Importa? Intenta sumar los precios:

SQL · EN DUCKDB

```
SELECT sum(precio_lista) FROM 'datos/erp/productos.csv';
```

ERROR

```
Binder Error: No function matches the given name and argument types 'sum(VARCHAR)'. You might need to
add explicit type casts.
        Candidate functions:
        sum(DECIMAL) -> DECIMAL
        sum(BOOLEAN) -> HUGEINT
        sum(SMALLINT) -> HUGEINT
        sum(INTEGER) -> HUGEINT
        sum(BIGINT) -> HUGEINT
        sum(HUGEINT) -> HUGEINT
        sum(DOUBLE) -> DOUBLE
        sum(BIGNUM) -> BIGNUM
```

Léelo por partes: *No function matches* significa "no hay ninguna versión de la función `sum` que acepte lo que le has dado"; `sum(VARCHAR)` es lo que le has dado: un texto. No se pueden sumar textos. Y debajo, DuckDB te sugiere añadir una conversión explícita (*explicit cast*).

IDEA CLAVE

Después de leer cualquier fichero, `DESCRIBE`. Un número leído como texto no da error al cargar: da error, o algo peor, un resultado incorrecto, semanas después, cuando alguien intenta sumarlo.

9.5 read_csv: decirle cómo leer

Cuando el detector no acierta, se lee con la función `read_csv`, que admite opciones para decirle exactamente cómo es el fichero:

SQL · EN DUCKDB

```
DESCRIBE SELECT * FROM read_csv('datos/erp/productos.csv', delim = ';', decimal_separator = ',');
```

RESULTADO

column_name varchar	column_type varchar	null varchar	key varchar	default varchar	extra varchar
producto_id	BIGINT	YES	NULL	NULL	NULL
sku	VARCHAR	YES	NULL	NULL	NULL
nombre	VARCHAR	YES	NULL	NULL	NULL
categoria	VARCHAR	YES	NULL	NULL	NULL
subcategoria	VARCHAR	YES	NULL	NULL	NULL
marca	VARCHAR	YES	NULL	NULL	NULL
formato	VARCHAR	YES	NULL	NULL	NULL
precio_lista	DOUBLE	YES	NULL	NULL	NULL
coste_unitario	DOUBLE	YES	NULL	NULL	NULL
iva_pct	BIGINT	YES	NULL	NULL	NULL
activo	VARCHAR	YES	NULL	NULL	NULL
fecha_alta	DATE	YES	NULL	NULL	NULL
fecha_baja	DATE	YES	NULL	NULL	NULL

13 rows 6 columns

Ahora `precio_lista` y `coste_unitario` son `DOUBLE`: números con decimales.

DESMONTANDO LA CONSULTA

`read_csv('datos/erp/productos.csv', delim = ';', decimal_separator = ',')`

`read_csv( ... )`	Una función que lee un CSV y devuelve una tabla. Va donde iría el nombre de una tabla, en el `FROM`.
`'datos/erp/productos.csv'`	El fichero. Entre comillas simples: es un texto.
`delim = ';'`	*Delimiter*: el separador de columnas. El detector ya lo había adivinado, pero decirlo explícitamente es más seguro: si un día el fichero llega raro, no adivinará otra cosa.
`decimal_separator = ','`	El carácter de los decimales. Es lo que faltaba.

LOS MILES

Esta opción funciona porque el ERP de Marejada escribe los números sin separador de miles: `1234,50`. Si llegase `1.234,50`, con el punto de los miles, DuckDB no podría leerlo como número. Es otra cosa que conviene mirar con `head` antes de cargar.

9.6 Fechas en formato español

Los pedidos tienen el mismo problema de los decimales, y otro más:

SQL · EN DUCKDB

```
DESCRIBE SELECT * FROM read_csv('datos/erp/pedidos.csv', delim = ';', decimal_separator = ',');
```

RESULTADO

column_name varchar	column_type varchar	null varchar	key varchar	default varchar	extra varchar
pedido_id	BIGINT	YES	NULL	NULL	NULL
fecha_pedido	VARCHAR	YES	NULL	NULL	NULL
cliente_id	BIGINT	YES	NULL	NULL	NULL
comercial_id	BIGINT	YES	NULL	NULL	NULL
canal	VARCHAR	YES	NULL	NULL	NULL
estado	VARCHAR	YES	NULL	NULL	NULL
fecha_entrega	DATE	YES	NULL	NULL	NULL
almacen	VARCHAR	YES	NULL	NULL	NULL
gastos_envio	DOUBLE	YES	NULL	NULL	NULL
observaciones	VARCHAR	YES	NULL	NULL	NULL

10 rows 6 columns

`fecha_entrega` sí la ha reconocido como `DATE`, pero `fecha_pedido` es `VARCHAR`. La diferencia es que `fecha_pedido` lleva hora (`01/01/2024 08:03`), y ese formato no lo ha reconocido. Se le dice cuál es:

SQL · EN DUCKDB

```
DESCRIBE SELECT * FROM read_csv('datos/erp/pedidos.csv', delim = ';', decimal_separator = ',',
                            timestampformat = '%d/%m/%Y %H:%M', dateformat = '%d/%m/%Y');
```

RESULTADO

column_name varchar	column_type varchar	null varchar	key varchar	default varchar	extra varchar
pedido_id	BIGINT	YES	NULL	NULL	NULL
fecha_pedido	TIMESTAMP	YES	NULL	NULL	NULL
cliente_id	BIGINT	YES	NULL	NULL	NULL
comercial_id	BIGINT	YES	NULL	NULL	NULL
canal	VARCHAR	YES	NULL	NULL	NULL
estado	VARCHAR	YES	NULL	NULL	NULL
fecha_entrega	DATE	YES	NULL	NULL	NULL
almacen	VARCHAR	YES	NULL	NULL	NULL
gastos_envio	DOUBLE	YES	NULL	NULL	NULL
observaciones	VARCHAR	YES	NULL	NULL	NULL

10 rows 6 columns

DESMONTANDO EL COMANDO

`timestampformat = '%d/%m/%Y %H:%M'`

`timestampformat`	El formato de las columnas con fecha y hora (en inglés, *timestamp*).
`%d`	El día, con dos cifras.
`%m`	El mes, con dos cifras. Ojo: `%m` minúscula es el mes; `%M` mayúscula son los minutos.
`%Y`	El año, con cuatro cifras.
`%H:%M`	Hora (de 00 a 23) y minutos.
`dateformat = '%d/%m/%Y'`	Lo mismo para las columnas que solo tienen fecha. El detector ya la había adivinado, pero así queda escrito y no depende de su suerte.

Estas letras con `%` son un estándar que comparten casi todos los lenguajes. Las volverás a ver en Python, en Spark y en Power BI.

EL ERROR QUE NO DA ERROR

Si el detector hubiera leído `04/01/2024` como mes/día, el 4 de enero se habría convertido en el 1 de abril, y no habría salido ningún error. Por eso, con fechas, nunca te fíes de la detección automática: escribe el formato.

9.7 El fichero en Latin-1

Ahora el de Recursos Humanos:

SQL · EN DUCKDB

```sql
SELECT * FROM 'datos/rrhh/comerciales.csv';
```

ERROR

```
Invalid Input Error: CSV Error on Line: 2
Original Line: comercial_id,nombre,apellidos,email,zona,puesto,jefe_id,fecha_alta,fecha_baja
Invalid unicode (byte sequence mismatch) detected. This file is not utf-8 encoded.
Possible Solution: Set the correct encoding, if available, to read this CSV File (e.g.,
encoding='UTF-16')
Possible Solution: Enable ignore errors (ignore_errors=true) to skip this row
```

Esta vez sí hay error, y es de los buenos, porque lo dice todo: *Invalid unicode (byte sequence mismatch)* significa "he encontrado bytes que no son UTF-8 válido"; y *This file is not utf-8 encoded* es la conclusión. La solución:

SQL · EN DUCKDB

```
SELECT comercial_id, nombre, apellidos, puesto
FROM read_csv('datos/rrhh/comerciales.csv', encoding = 'latin-1')
LIMIT 4;
```

RESULTADO

comercial_id	nombre	apellidos	puesto
int64	varchar	varchar	varchar
1	Íñigo	Sainz Ruiz	Director comercial
2	Raquel	Fernández Cobo	Jefa de ventas Cantabria
3	Daniel	Gutiérrez Pérez	Comercial
4	Sara	Ceballos Ortiz	Comercial

`Íñigo` aparece bien. DuckDB ha leído los bytes con la tabla correcta y los ha convertido a UTF-8 por dentro. A partir de aquí, ya no hay ningún problema de codificación.

9.8 Crear el esquema y las tablas

Hasta ahora has leído ficheros, pero no has guardado nada en la base de datos. Primero, un sitio donde guardarlo:

SQL · EN DUCKDB

```
CREATE SCHEMA IF NOT EXISTS bronze;
```

Un **esquema** es una carpeta dentro de la base de datos, para agrupar tablas. Tendrás tres: `bronze`, `silver` y más adelante `gold`, como las capas de 9.1. El `IF NOT EXISTS` evita un error si ya existía: "créalo, salvo que ya exista".

Y ahora, la primera tabla:

SQL · EN DUCKDB

```
CREATE OR REPLACE TABLE bronze.clientes AS
SELECT * FROM read_csv('datos/erp/clientes.csv', delim = ';', decimal_separator = ',');
```

No sale nada, como en la terminal cuando un comando funciona.

DESMONTANDO LA CONSULTA

`CREATE OR REPLACE TABLE bronze.clientes AS SELECT ...`

`CREATE TABLE`	Crea una tabla nueva en la base de datos.
`OR REPLACE`	Y si ya existe, la sustituye. Así puedes ejecutarlo veinte veces sin error, y siempre quedará la versión de la última vez.
`bronze.clientes`	El nombre: esquema, punto, tabla. Como una ruta: "la tabla `clientes` dentro de `bronze`".
`AS SELECT ...`	"Con el resultado de esta consulta". Las columnas, sus tipos y sus filas salen de lo que devuelva el `SELECT`.

Compruébala:

SQL · EN DUCKDB

```
SELECT count(*) AS filas FROM bronze.clientes;
```

RESULTADO

```
┌─────────┐
│ filas   │
│ int64   │
├─────────┤
│    1212 │
└─────────┘
```

1.212: las 1.213 líneas del fichero menos la cabecera. Coincide.

9.9 Todo en un script

Podrías crear las otras cinco tablas escribiendo cada `CREATE` en DuckDB. Pero eso tiene el mismo problema que hacer las cosas con el ratón: no queda escrito. Si mañana llega una versión nueva de los datos, o borras la base de datos sin querer, tendrías que acordarte de cada opción de cada fichero.

La forma profesional es escribir la carga en un **script**: un fichero `.sql` con todas las sentencias, que se ejecuta de una vez. Sal de DuckDB con `.quit`, crea la carpeta del volumen y el fichero:

TÚ ESCRIBES

```
mkdir -p sql/01-puesta-a-punto
code sql/00_cargar_bronze.sql
```

FICHERO · sql/00_cargar_bronze.sql

```
-- ============================================================
-- Carga de la capa BRONZE de Marejada Distribución
-- Copia fiel de los ficheros de origen, solo con los tipos
-- bien leídos. Aquí no se corrige ningún dato.
-- Uso:  duckdb db/marejada.duckdb < sql/00_cargar_bronze.sql
-- ============================================================

CREATE SCHEMA IF NOT EXISTS bronze;

-- ERP: separador ";" y coma decimal
CREATE OR REPLACE TABLE bronze.clientes AS
SELECT * FROM read_csv('datos/erp/clientes.csv',
    delim = ';', decimal_separator = ',');

CREATE OR REPLACE TABLE bronze.productos AS
SELECT * FROM read_csv('datos/erp/productos.csv',
    delim = ';', decimal_separator = ',');

-- Los pedidos traen las fechas en formato español
CREATE OR REPLACE TABLE bronze.pedidos AS
SELECT * FROM read_csv('datos/erp/pedidos.csv',
    delim = ';', decimal_separator = ',',
    timestampformat = '%d/%m/%Y %H:%M', dateformat = '%d/%m/%Y');

CREATE OR REPLACE TABLE bronze.lineas_pedido AS
SELECT * FROM read_csv('datos/erp/lineas_pedido.csv',
    delim = ';', decimal_separator = ',');

-- RRHH: programa antiguo, codificación Latin-1
CREATE OR REPLACE TABLE bronze.comerciales AS
SELECT * FROM read_csv('datos/rrhh/comerciales.csv',
    encoding = 'latin-1');

-- Almacén: CSV estándar
CREATE OR REPLACE TABLE bronze.inventario AS
SELECT * FROM read_csv('datos/almacen/inventario_2026-09-30.csv');
```

Fíjate en tres detalles del script. Las líneas que empiezan por `--` son **comentarios** de SQL, como el `#` de la terminal. El de arriba explica qué hace el script y cómo se usa, que es lo primero que alguien querrá saber al abrirlo. Y todas las tablas usan `CREATE OR REPLACE`: el script se puede ejecutar las veces que haga falta y siempre deja la base de datos en el mismo estado. A eso se le llama que un proceso sea **idempotente**, y es una de las propiedades más importantes de cualquier carga de datos.

Ejecútalo desde la terminal:

TÚ ESCRIBES

```
duckdb db/marejada.duckdb < sql/00_cargar_bronze.sql
```

DESMONTANDO EL COMANDO

`duckdb db/marejada.duckdb < sql/00_cargar_bronze.sql`

`duckdb db/marejada.duckdb`	Abre la base de datos, como siempre.
`< sql/00_cargar_bronze.sql`	La redirección de entrada del Volumen 1: en lugar de esperar a que escribas, DuckDB lee las órdenes del fichero. Las ejecuta todas y termina.

No dice nada y vuelve el prompt de la terminal en un par de segundos. El silencio, otra vez, es buena señal. Dentro de DuckDB, lo mismo se hace con `.read sql/00_cargar_bronze.sql`.

9.10 Comprobar la carga

Entra en la base de datos y cuenta las filas de cada tabla:

TÚ ESCRIBES

```
duckdb db/marejada.duckdb
```

SQL · EN DUCKDB

```
SELECT 'clientes' AS tabla, count(*) AS filas FROM bronze.clientes
UNION ALL SELECT 'productos', count(*) FROM bronze.productos
UNION ALL SELECT 'pedidos', count(*) FROM bronze.pedidos
UNION ALL SELECT 'lineas_pedido', count(*) FROM bronze.lineas_pedido
UNION ALL SELECT 'comerciales', count(*) FROM bronze.comerciales
UNION ALL SELECT 'inventario', count(*) FROM bronze.inventario;
```

RESULTADO

```
┌─────────────────┬─────────┐
│     tabla       │ filas   │
│    varchar      │ int64   │
├─────────────────┼─────────┤
│ clientes        │    1212 │
│ productos       │     235 │
│ pedidos         │   77448 │
│ lineas_pedido   │  381158 │
│ comerciales     │      14 │
│ inventario      │     381 │
└─────────────────┴─────────┘
```

`UNION ALL` pega los resultados de varias consultas uno debajo de otro. Compara con el `wc -l` de 8.4, restando la cabecera: 1.213 − 1 = 1.212 clientes, 381.159 − 1 = 381.158 líneas. Cuadra todo.

> **IDEA CLAVE**
>
> Una carga no está terminada cuando no da errores: está terminada cuando has comprobado que han llegado todas las filas y con los tipos correctos. Contar filas en origen y en destino es la comprobación más básica, y la que más problemas detecta.

9.11 Ver lo que hay

El comando con punto rápido:

TÚ ESCRIBES

```
.tables
```

RESULTADO

```
──────────────────────── marejada ────────────────────────
──────────────────────── bronze ─────────────────────────
┌──────────────────────────┐┌──────────────────────────┐
│         clientes         ││        productos         │
│                          ││                          │
│ cliente_id         bigint ││ producto_id     bigint   │
│ nif                varchar ││ sku             varchar │
│ nombre_comercial   varchar ││ nombre          varchar │
│ ...                      ││ ...                      │
│                          ││                          │
│        1212 rows         ││        235 rows          │
└──────────────────────────┘└──────────────────────────┘
   ... y así con las seis tablas
```

En las versiones recientes de DuckDB, `.tables` dibuja cada tabla como una tarjeta, con sus columnas, sus tipos y su número de filas, agrupadas por base de datos (`marejada`) y por esquema (`bronze`). En tu terminal sale en colores y con las columnas completas; aquí está recortado para que quepa.

Y la versión SQL, que da más información. DuckDB guarda una descripción de todo lo que contiene la base de datos, y se consulta como una tabla más:

SQL · EN DUCKDB

```sql
SELECT schema_name, table_name, column_count, estimated_size
FROM duckdb_tables()
ORDER BY table_name;
```

RESULTADO

```
┌─────────────┬──────────────┬──────────────┬────────────────┐
│ schema_name │  table_name  │ column_count │ estimated_size │
│   varchar   │    varchar   │    int64     │     int64      │
├─────────────┼──────────────┼──────────────┼────────────────┤
│ bronze      │ clientes     │           15 │           1212 │
│ bronze      │ comerciales  │            9 │             14 │
│ bronze      │ inventario   │            6 │            381 │
│ bronze      │ lineas_pedido│            7 │         381158 │
│ bronze      │ pedidos      │           10 │          77448 │
│ bronze      │ productos    │           13 │            235 │
└─────────────┴──────────────┴──────────────┴────────────────┘
```

`estimated_size` es el número aproximado de filas. A estas tablas sobre la propia base de datos se les llama **metadatos**: datos sobre los datos.

10. La misma base de datos desde Python

10.1 Crear un notebook

Un **notebook** es un documento en el que se mezclan celdas de código y su resultado, una debajo de otra. Es la forma estándar de explorar datos en Python. Los ficheros tienen extensión `.ipynb` (de *IPython notebook*, el nombre antiguo de Jupyter).

Abre el proyecto en VS Code y crea el notebook:

TÚ ESCRIBES

```
code .
```

1. Pulsa `Cmd` + `Shift` + `P` y escribe `Create: New Jupyter Notebook`.
2. Guárdalo con `Cmd` + `S` dentro de `notebooks/`, con el nombre `01_comprobacion.ipynb`.

10.2 Elegir el Python del proyecto

Arriba a la derecha del notebook verás un botón que dice **Select Kernel**. El *kernel* es el Python que ejecuta las celdas. Púlsalo y elige **Python Environments** y después el que lleva `.venv` en el nombre: el del proyecto.

> **SI NO APARECE EL .VENV**
>
> Cierra VS Code y ábrelo de nuevo desde la carpeta del proyecto con `code .`. VS Code busca entornos virtuales en la carpeta abierta; si abriste solo el fichero, no lo encuentra.

10.3 Consultar desde Python

En la primera celda escribe esto y ejecútala con `Shift` + `Intro`:

PYTHON · EN TU NOTEBOOK

```
import duckdb

con = duckdb.connect("../db/marejada.duckdb", read_only=True)
con.sql("SELECT estado, count(*) AS pedidos FROM bronze.pedidos GROUP BY estado ORDER BY pedidos DESC")
```

RESULTADO

```
┌──────────────────┬──────────┐
│      estado      │ pedidos  │
│     varchar      │  int64   │
├──────────────────┼──────────┤
│ Entregado        │    73280 │
│ Cancelado        │     2732 │
│ Devuelto parcial │     1164 │
│ En reparto       │      140 │
│ Pendiente        │      132 │
└──────────────────┴──────────┘
```

DESMONTANDO EL COMANDO

```
con = duckdb.connect("../db/marejada.duckdb", read_only=True)
```

`import duckdb`	Carga el paquete de DuckDB que instalaste con uv.
`duckdb.connect( ... )`	Abre la base de datos y devuelve una **conexión**: el canal por el que le harás las consultas.
`"../db/marejada.duckdb"`	La ruta, relativa al notebook. El notebook está en `notebooks/`, así que hay que subir un nivel (`..`) para llegar a `db/`.
`read_only=True`	Abrirla solo para leer. Ahora verás por qué.
`con.sql("...")`	Ejecuta una consulta. El resultado se pinta con el mismo formato de cajas que en la terminal.

Es la misma base de datos, la misma consulta y el mismo resultado que en la terminal. Y para que el resultado sea una tabla de Python con la que seguir trabajando, se convierte a pandas con `.df()`:

PYTHON · EN TU NOTEBOOK

```
pedidos = con.sql("SELECT * FROM bronze.pedidos").df()
pedidos.shape
```

RESULTADO

```
(77448, 10)
```

`shape` es la forma de la tabla: 77.448 filas y 10 columnas. Lo que puedes hacer con esa tabla es todo el Volumen 06. Hoy basta con saber que el camino está abierto.

10.4 El error del candado

Si tienes DuckDB abierto en la terminal con la base de datos y a la vez la abres desde Python **sin** `read_only=True`, verás esto:

ERROR

```
IOException: IO Error: Could not set lock on file "/Users/alvaro/master-data-ia/db/marejada.duckdb":
Conflicting lock is held in /opt/homebrew/bin/duckdb (PID 51877).
See also https://duckdb.org/docs/stable/connect/concurrency
```

Una base de datos DuckDB solo puede tener **un programa escribiendo** a la vez. Si dos programas escribieran en el mismo fichero al mismo tiempo, podrían estropearlo. Para evitarlo, el primero que la abre para escribir pone un **candado** (*lock*) en el fichero, y el segundo se encuentra la puerta cerrada. Fíjate en que el mensaje te dice quién tiene el candado: el programa (`/opt/homebrew/bin/duckdb`, la terminal) y su `PID`, como en el capítulo 9 del Volumen 1.

Soluciones:

- Cerrar la otra sesión (`.quit` en DuckDB) antes de abrirla desde Python.
- Abrirla en los dos sitios con permiso solo de lectura: en Python con `read_only=True`, y en la terminal con `duckdb -readonly db/marejada.duckdb`. Varios lectores a la vez no tienen problema.

La costumbre que te recomiendo: **los notebooks, siempre** `read_only=True`. Las tablas se crean con los scripts `.sql`, y los notebooks solo leen. Así nunca hay conflicto.

11. Guardar y publicar

TÚ ESCRIBES

```
git status
```

RESULTADO

```
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        notebooks/01_comprobacion.ipynb
        sql/00_cargar_bronze.sql

nothing added to commit but untracked files present (use "git add" to track)
```

Otra vez, la comprobación importante es lo que **no** aparece: ni `datos/` ni `db/marejada.duckdb`. Tu script sí, que es lo valioso.

TÚ ESCRIBES

```
git add sql/00_cargar_bronze.sql
git commit -m "feat: script de carga de la capa bronze"
git add notebooks/01_comprobacion.ipynb
git commit -m "feat: notebook de comprobación de la conexión desde Python"
git push
```

Dos commits porque son dos cosas distintas, como en el Volumen 1.

12. Una vista visual (opcional)

DuckDB trae una interfaz gráfica en el navegador, por si en algún momento prefieres ver los resultados de forma más cómoda:

TÚ ESCRIBES

```
duckdb -ui db/marejada.duckdb
```

Se abre una pestaña del navegador con un editor de consultas tipo notebook y la lista de tablas a la izquierda. La primera vez descarga un componente, así que necesita internet. Se cierra con `Ctrl` + `C` en la terminal.

Es cómoda para explorar, pero en los volúmenes trabajaremos siempre en la terminal: así los resultados que veas serán idénticos a los del libro, y te acostumbras a la herramienta que vas a encontrar en cualquier servidor.

TICKETS

Tus primeros encargos

Cuatro tickets de bienvenida. Pequeños, pero reales: es lo que te pedirían la primera semana en cualquier equipo de datos.

Tickets · Volumen 01

Soluciones · Volumen 01

Tickets · Volumen 01

Para cada ticket: crea su fichero en `sql/01-puesta-a-punto/` (o donde diga el entregable), resuélvelo y haz commit. Intenta todos antes de mirar las soluciones.

TICKET MJD-100 De: Elena Pérez · Responsable de datos

Prioridad: alta Plazo: hoy

Bienvenido, Álvaro. Antes de que nadie te pida nada, necesito que conozcas los datos. Hazme un inventario de lo que hay en la capa bronze.

CRITERIOS DE ACEPTACIÓN

- Una fila por tabla, con su número de filas y de columnas.
- Para los pedidos, la fecha del primer pedido y la del último.
- Para la tienda web: cuántos pedidos tiene el JSON y entre qué fechas (todavía no lo hemos cargado; léelo directamente del fichero).

ENTREGABLE

`entregables/01_inventario_datos.md`, un documento corto con una tabla y las cifras. Y las consultas que hayas usado, en `sql/01-puesta-a-punto/mjd-100_inventario.sql`.

TICKET MJD-101 De: Elena Pérez · Responsable de datos

Prioridad: media

Quiero estar segura de que la base de datos se puede reconstruir desde cero en cualquier momento. Bórrala, reconstrúyela solo con tu script y dime cuánto tarda.

CRITERIOS DE ACEPTACIÓN

- La base de datos reconstruida tiene las mismas filas que antes en cada tabla.
- Me dices el tiempo.

TICKET MJD-102 De: Marcos Diego · Logística y almacén

Prioridad: media

Hola. Me han dicho que ya tienes el inventario del 30 de septiembre cargado. ¿Me dices cuántas referencias distintas tenemos en cada almacén y cuántas unidades en total en cada uno?

TICKET MJD-103 De: Elena Pérez · Responsable de datos

Prioridad: baja

Añade al README del repositorio una sección "Datos" con una frase sobre cada fuente: de qué sistema viene y en qué formato llega. Lo que has aprendido en el capítulo 8, para quien llegue detrás.

Soluciones · Volumen 01

MJD-100 · Inventario de datos

Las filas de cada tabla ya las contaste en 9.10. Las columnas, con los metadatos de 9.11:

SQL · EN DUCKDB

```
SELECT table_name AS tabla, estimated_size AS filas, column_count AS columnas
FROM duckdb_tables()
WHERE schema_name = 'bronze'
ORDER BY table_name;
```

RESULTADO

```
┌─────────────────┬─────────┬─────────────┐
│     tabla       │ filas   │ columnas    │
│    varchar      │ int64   │  int64      │
├─────────────────┼─────────┼─────────────┤
│ clientes        │    1212 │          15 │
│ comerciales     │      14 │           9 │
│ inventario      │     381 │           6 │
│ lineas_pedido   │  381158 │           7 │
│ pedidos         │   77448 │          10 │
│ productos       │     235 │          13 │
└─────────────────┴─────────┴─────────────┘
```

ESTIMATED SIGNIFICA ESTIMADO

`estimated_size` es una estimación que DuckDB guarda para planificar consultas. En tablas recién cargadas suele coincidir con la realidad, pero no está garantizado. Para un entregable, el número de filas se saca con `count(*)`, como en 9.10. Los metadatos sirven para el número de columnas, que sí es exacto.

El rango de fechas de los pedidos:

SQL · EN DUCKDB

```
SELECT min(fecha_pedido) AS primer_pedido, max(fecha_pedido) AS ultimo_pedido
FROM bronze.pedidos;
```

RESULTADO

```
┌─────────────────────┬─────────────────────┐
│    primer_pedido    │    ultimo_pedido    │
│      timestamp      │      timestamp      │
├─────────────────────┼─────────────────────┤
│ 2024-01-01 08:03:00 │ 2026-09-30 23:32:00 │
└─────────────────────┴─────────────────────┘
```

Y el JSON de la tienda web, sin cargarlo. Primero, ver su forma:

SQL · EN DUCKDB

```
DESCRIBE SELECT * FROM 'datos/web/pedidos_web.json';
```

RESULTADO

column_name varchar	column_type varchar	null varchar	key varchar	default varchar	extra varchar
source	VARCHAR	YES	NULL	NULL	NULL
exported_at	TIMESTAMP	YES	NULL	NULL	NULL
count	BIGINT	YES	NULL	NULL	NULL
orders	STRUCT(order_id VARCHAR, created_at TIMES…	YES	NULL	NULL	NULL

El fichero no es una lista de pedidos, sino un objeto con cuatro campos: `source`, `exported_at`, `count` y `orders`, que es una lista (fíjate en los `[]` del final de su tipo) con un pedido dentro de cada elemento. Cada pedido, a su vez, tiene campos dentro (`STRUCT`), y uno de ellos, `items`, es otra lista. Es un JSON **anidado**: datos dentro de datos. Lo trabajarás a fondo en el Volumen 06. Para este ticket basta con desplegar la lista:

SQL · EN DUCKDB

```
SELECT count(*) AS pedidos_web, min(o.created_at) AS primero, max(o.created_at) AS ultimo
FROM (SELECT unnest(orders) AS o FROM 'datos/web/pedidos_web.json');
```

RESULTADO

pedidos_web int64	primero timestamp	ultimo timestamp
7956	2025-04-07 04:04:00	2026-09-30 21:32:00

DESMONTANDO LA CONSULTA

`SELECT ... FROM (SELECT unnest(orders) AS o FROM 'datos/web/pedidos_web.json')`

`unnest(orders)`	"Despliega la lista": convierte una fila que contiene una lista de 7.956 pedidos en 7.956 filas, una por pedido.
`AS o`	Cada pedido desplegado se llama `o`.
`( ... )`	La consulta de dentro se usa como si fuera una tabla. Es una **subconsulta**, y la verás a fondo en el Volumen 03.
`o.created_at`	El campo `created_at` de cada pedido. El punto entra dentro de una estructura, igual que en los objetos de JavaScript del Volumen 1.

El entregable:

FICHERO · entregables/01_inventario_datos.md

```
# Inventario de datos · capa bronze

Fecha: 19/10/2026 · Autor: Álvaro

| Tabla          | Filas   | Columnas | Origen                   |
|----------------|---------|----------|--------------------------|
| clientes       | 1.212   | 15       | ERP                      |
| comerciales    | 14      | 9        | RRHH                     |
| inventario     | 381     | 6        | Almacén                  |
| lineas_pedido  | 381.158 | 7        | ERP                      |
| pedidos        | 77.448  | 10       | ERP                      |
| productos      | 235     | 13       | ERP                      |

- Pedidos del ERP: del 01/01/2024 al 30/09/2026.
- Tienda web (JSON, aún sin cargar): 7.956 pedidos,
  del 07/04/2025 al 30/09/2026 (horas en UTC).
```

REVISIÓN DE ELENA

Bien. Dos cosas que me gustan y que conviene que hagas siempre. La primera, poner la fecha y el autor: dentro de tres meses alguien abrirá esto y querrá saber de cuándo son las cifras. La segunda, haber apuntado que las horas del JSON están en UTC (la `Z` del final en el fichero original). Es la hora universal, no la española: en verano, las 04:04 UTC son las 06:04 en Santander. Cuando crucemos la web con el ERP, esa diferencia importará.

Y una pregunta para que la pienses, sin necesidad de contestarla todavía: el ERP tiene pedidos con canal "Web B2B". ¿Serán los mismos 7.956 del JSON?

MJD-101 · Reconstruir desde cero

TÚ ESCRIBES

```
rm db/marejada.duckdb
time duckdb db/marejada.duckdb < sql/00_cargar_bronze.sql
```

RESULTADO

```
real    0m1.43s
user    0m3.10s
sys     0m0.35s
```

`time` es un comando de la terminal que ejecuta lo que le pongas detrás y al terminar te dice cuánto ha tardado. La línea que te interesa es `real`: el tiempo de reloj. Unos pocos segundos: cargar 25 MB de CSV en DuckDB es muy rápido. Después, repite el recuento de filas de 9.10 y comprueba que coincide.

¿Por qué `user` puede salir mayor que `real`? Porque `user` suma el tiempo de todos los núcleos del procesador. DuckDB reparte el trabajo entre varios núcleos a la vez, así que en un segundo de reloj puede gastar dos o tres segundos de procesador.

REVISIÓN DE ELENA

Esto que acabas de hacer parece una tontería y es de lo más importante del día. Una base de datos que solo se puede reconstruir "a mano" acaba siendo una base de datos que nadie se atreve a tocar. Una que se reconstruye con un comando en segundos se puede tirar y rehacer cada vez que algo huela raro.

MJD-102 · Referencias y unidades por almacén

SQL · EN DUCKDB

```
SELECT almacen,
       count(DISTINCT sku) AS referencias,
       sum(unidades)       AS unidades
FROM bronze.inventario
GROUP BY almacen
ORDER BY almacen;
```

RESULTADO

```
┌──────────┬────────────┬──────────┐
│ almacen  │ referencias │ unidades │
│ varchar  │    int64    │ int128   │
├──────────┼────────────┼──────────┤
│ Guarnizo │        235 │    13319 │
│ Llanera  │        146 │     8130 │
└──────────┴────────────┴──────────┘
```

Es SQL de ASIR: agrupar por almacén y, en cada grupo, contar y sumar. `count(DISTINCT sku)` cuenta los SKU distintos, no las filas, que es lo que pedía Marcos ("referencias distintas").

REVISIÓN DE ELENA

El número es correcto según los datos... pero no se lo mandaría a Marcos todavía. Mira las referencias con esto: `SELECT sku FROM bronze.inventario ORDER BY sku LIMIT 20;`. Verás que algunos SKU vienen en minúsculas, con un espacio delante o sin guion: `cer-0012`, `CER-0012` y `CER0012` son la misma referencia escrita de tres formas, y `count(DISTINCT ...)` las cuenta como tres. Arreglarlo es parte del Volumen 02. Por ahora, la lección: un número que sale de datos sin revisar se entrega con la advertencia, o no se entrega.

MJD-103 · El README

FICHERO · README.md (sección nueva)

```
## Datos

Los datos son de Marejada Distribución (empresa ficticia) y no se
suben al repositorio. Llegan de cinco sistemas:

- **ERP** (`datos/erp/`): clientes, productos, pedidos y líneas.
  CSV con separador `;`, coma decimal y fechas dd/mm/aaaa.
- **RRHH** (`datos/rrhh/`): comerciales. CSV en Latin-1.
- **Almacén** (`datos/almacen/`): inventario a 30/09/2026. CSV estándar.
- **Contabilidad** (`datos/contabilidad/`): facturación mensual en Excel.
- **Tienda web** (`datos/web/`): pedidos online en JSON anidado, horas en UTC.

La capa `bronze` se carga con `sql/00_cargar_bronze.sql`.
```

TÚ ESCRIBES

```
git add README.md
git commit -m "docs: describe las fuentes de datos en el README"
git push
```

APÉNDICES

Para tener a mano

El diccionario de datos de Marejada, una chuleta de lo nuevo y lo que viene después.

A. Diccionario de datos

B. Chuleta

C. Qué viene en el Volumen 02

A. Diccionario de datos

Lo que dice la documentación de cada sistema. Es la versión oficial: lo que los datos **deberían** ser. Lo que son de verdad lo irás descubriendo en los tickets.

bronze.clientes (ERP)

COLUMNA	DESCRIPCIÓN
cliente_id	Identificador del cliente en el ERP.
nif	NIF o CIF del cliente.
nombre_comercial	Nombre del local.
razon_social	Nombre legal de la sociedad o persona titular.
tipo_cliente	Bar, Restaurante, Cafetería, Hotel, Supermercado, Tienda de alimentación, Club / Discoteca, Catering o Sidrería.
municipio, provincia, codigo_postal	Dirección de entrega.
telefono, email	Contacto.
comercial_id	Comercial asignado actualmente.
fecha_alta	Fecha de alta como cliente.
activo	S o N.
limite_credito	Crédito máximo concedido, en euros.
descuento_habitual	Descuento por defecto del cliente, en %.

bronze.productos (ERP)

COLUMNA	DESCRIPCIÓN
producto_id	Identificador del producto en el ERP.
sku	Código de referencia, compartido con el almacén: tres letras de la categoría, guion y cuatro cifras.
nombre, marca, formato	Descripción del artículo y su presentación (caja, barril, botella...).
categoria, subcategoria	Clasificación comercial.
precio_lista	Precio de tarifa vigente (2026), en euros, sin IVA.
coste_unitario	Coste de compra, en euros.
iva_pct	Tipo de IVA aplicable, en %.
activo, fecha_alta, fecha_baja	Si se vende actualmente y desde/hasta cuándo.

bronze.pedidos (ERP)

COLUMNA	DESCRIPCIÓN
pedido_id	Identificador del pedido: año y número correlativo.
fecha_pedido	Fecha y hora en que se registró el pedido.
cliente_id, comercial_id	Quién lo pide y qué comercial lo lleva.
canal	Comercial (visita), Teléfono o Web B2B.
estado	Pendiente, En reparto, Entregado, Devuelto parcial o Cancelado.
fecha_entrega	Fecha de entrega al cliente.
almacen	Guarnizo o Llanera.
gastos_envio	Portes cobrados, en euros sin IVA.
observaciones	Texto libre.

bronze.lineas_pedido (ERP)

COLUMNA	DESCRIPCIÓN
linea_id	Identificador de la línea.
pedido_id, num_linea	Pedido al que pertenece y número de línea dentro del pedido.
producto_id	Producto.
cantidad	Unidades de venta (cajas, barriles...). Las devoluciones se registran como líneas en negativo.
precio_unitario	Precio aplicado en la fecha del pedido, en euros sin IVA.
descuento_pct	Descuento de la línea, en %.

bronze.comerciales (RRHH)

COLUMNA	DESCRIPCIÓN
comercial_id	Identificador; coincide con `comercial_id` del ERP.
nombre, apellidos, email	Datos de la persona.
zona	Zona comercial que lleva.
puesto, jefe_id	Cargo y a quién reporta (otro `comercial_id`).
fecha_alta, fecha_baja	Entrada y salida de la empresa.

bronze.inventario (almacén)

COLUMNA	DESCRIPCIÓN
almacen	Guarnizo o Llanera.

COLUMNA	DESCRIPCIÓN
sku	Referencia del producto.
descripcion	Texto descriptivo del programa de almacén.
unidades	Unidades contadas en el inventario.
ubicacion	Pasillo, estantería y hueco.
fecha_inventario	Fecha del recuento.

B. Chuleta

DuckDB en la terminal

COMANDO	QUÉ HACE
`duckdb`	Sesión en memoria (se pierde al salir).
`duckdb db/marejada.duckdb`	Abre (o crea) esa base de datos.
`duckdb -readonly db/marejada.duckdb`	La abre solo para leer.
`duckdb db/x.duckdb < script.sql`	Ejecuta un script y sale.
`duckdb -ui db/x.duckdb`	Interfaz visual en el navegador.
`.tables` · `.timer on` · `.read f.sql` · `.quit`	Tablas · cronómetro · ejecutar script · salir

Leer ficheros en DuckDB

NECESITAS	ESCRIBE
Ver un CSV sin cargarlo	`SELECT * FROM 'ruta.csv' LIMIT 5;`
Ver los tipos que ha deducido	`DESCRIBE SELECT * FROM 'ruta.csv';`
Separador punto y coma	`read_csv('ruta.csv', delim = ';')`
Coma decimal	`decimal_separator = ','`
Fechas día/mes/año	`dateformat = '%d/%m/%Y'`
Fecha y hora	`timestampformat = '%d/%m/%Y %H:%M'`
Codificación antigua	`encoding = 'latin-1'`
Guardarlo como tabla	`CREATE OR REPLACE TABLE esquema.tabla AS SELECT ...;`

uv

COMANDO	QUÉ HACE
`uv init --bare --python 3.12`	Convierte la carpeta en proyecto de Python.
`uv add paquete`	Instala un paquete en el proyecto y lo apunta.
`uv remove paquete`	Lo quita.
`uv sync`	Recrea el entorno exacto a partir de `uv.lock`.
`uv run comando`	Ejecuta algo con el Python del proyecto.

C. Qué viene en el Volumen 02

El Volumen 02 retoma el SQL de ASIR y lo lleva al nivel de un analista, siempre sobre la capa bronze que acabas de cargar:

- Consultar y filtrar, con las trampas de los nulos y las fechas.
- Transformar columnas: textos, números, fechas y `CASE`.
- Agregar sin equivocarse de nivel de detalle.
- Unir tablas, y la trampa que hace que las sumas salgan multiplicadas.
- Limpiar datos con SQL: duplicados, nombres escritos de veinte formas, fechas imposibles.
- Construir la capa silver.
- Diez tickets de los distintos departamentos de Marejada, incluido el de Lucía: la facturación de un mes que no cuadra con contabilidad.

ANTES DE PASAR AL VOLUMEN 02

Tienes que poder hacer esto sin mirar: abrir la base de datos desde la carpeta del proyecto, reconstruirla con el script, ver los tipos de una tabla con `DESCRIBE` y consultarla desde un notebook. Si algo de eso falla, este es el momento de arreglarlo, no a mitad de un ticket.